

Skip List – Insert Element

–Program Analysis

–Insertion

(Attempting to insert a duplicate key)

*SkipList * list*

Address *list*

ff33 *ff08*

Address *level* ** header*

ff08 **3** *ff12*

Address *key* ** forward*

ff12 -1 *ff80*

Address *Values*

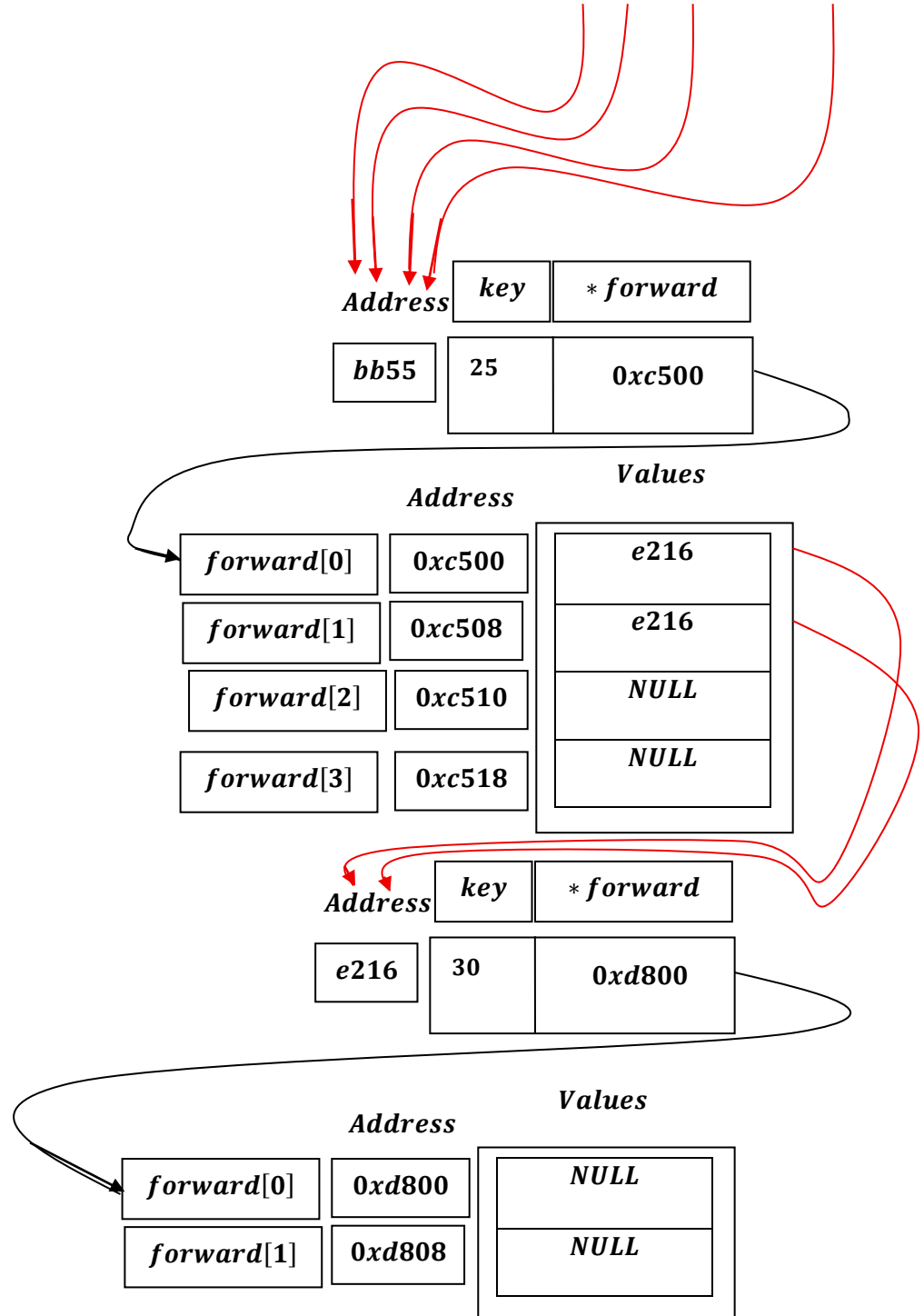
<i>forward[0]</i>	<i>ff80</i>	<i>ff16</i>
<i>forward[1]</i>	<i>ff88</i>	<i>ff16</i>
<i>forward[2]</i>	<i>ff90</i>	<i>ff16</i>
<i>forward[3]</i>	<i>ff98</i>	<i>bb55</i>

Address *key* ** forward*

ff16 20 *0xb400*

Address *Values*

<i>forward[0]</i>	<i>0xb400</i>	<i>bb55</i>
<i>forward[1]</i>	<i>0xb408</i>	<i>bb55</i>
<i>forward[2]</i>	<i>0xb410</i>	<i>bb55</i>



```
case 1:
    cout << "Enter key to insert: ";
    cin >> key;
    insertElement(list, key);
    break;
```

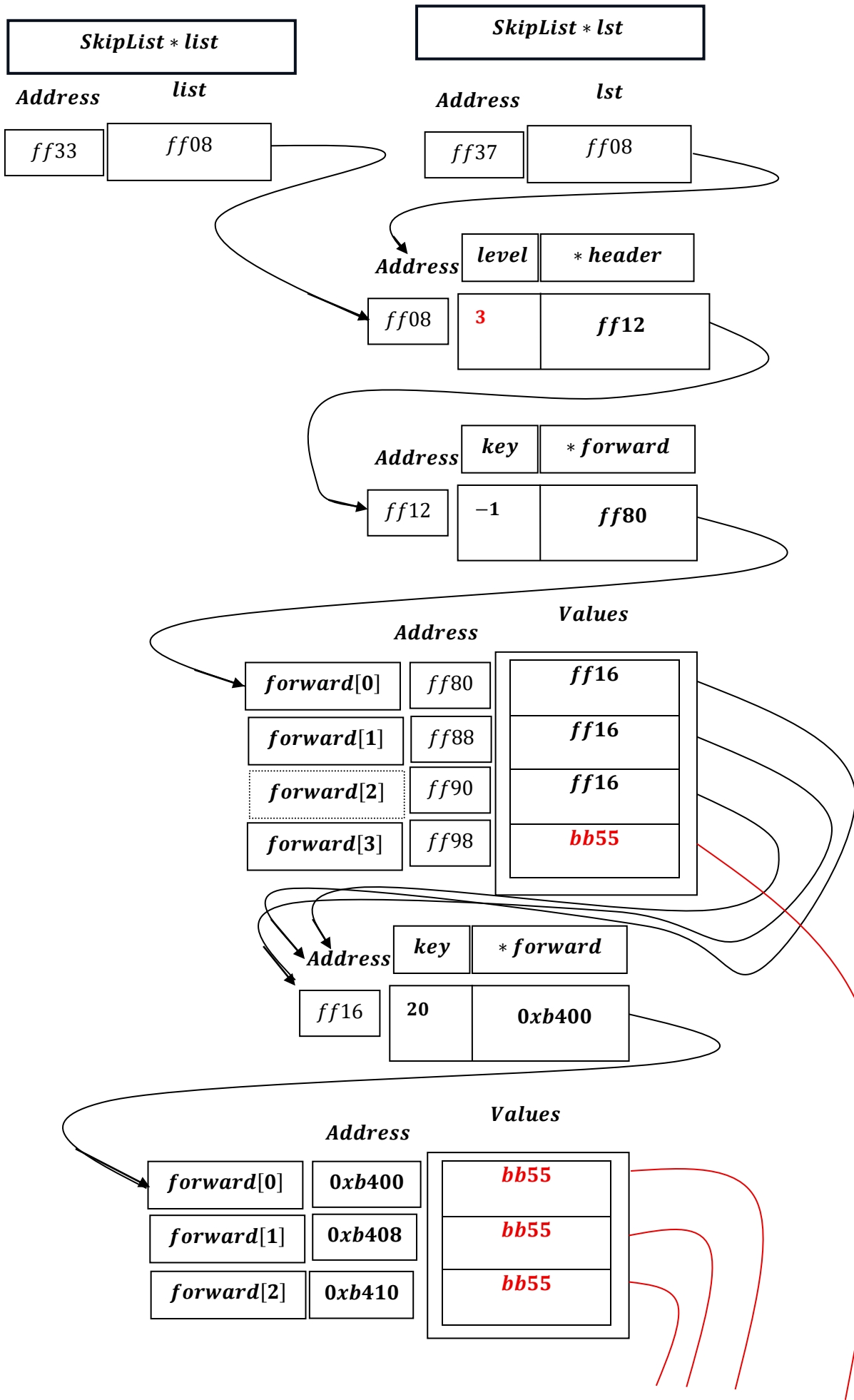
Entering : Key = 25 .

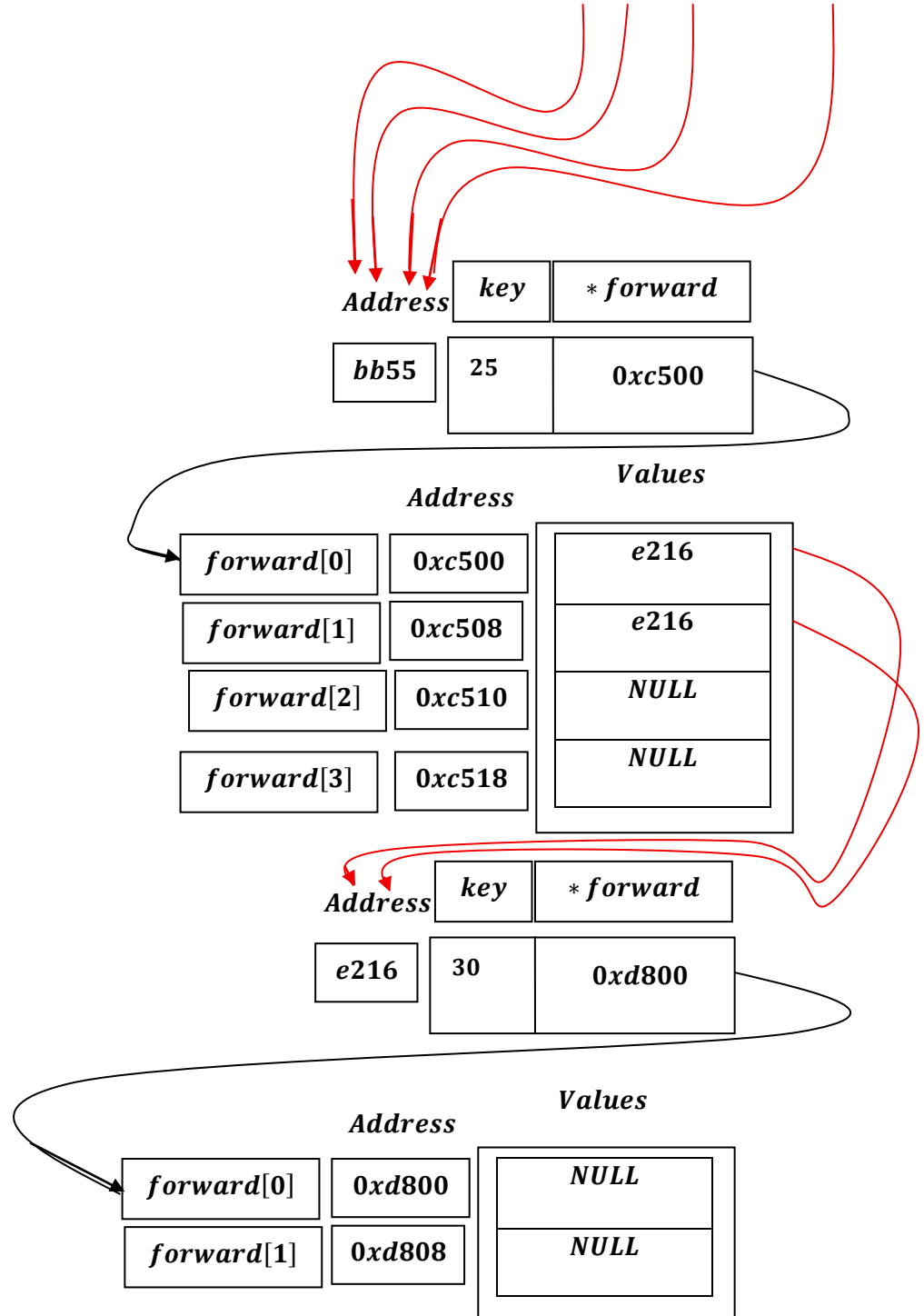
insertElement(list: ff08 ,key: 25); [25 is a duplicate key]

Now ,we have :

```
void insertElement(struct SkipList *lst, int key)
```

*list = struct SkipList * lst = ff08 and int key: 25.*





```
struct Node *current = lst->header;
```

current = *lst* → *header* : *ff12*.

```
struct Node *update[MAX_LEVEL + 1];
```

update[3 + 1 = 4],

MAX_LEVEL evaluates to 3. Consequently, the array size calculation is $\text{update}[3 + 1 = 4]$.

This means the array is allocated to hold exactly 4 elements.

Before memset call, the update array may look like :

<i>Array Element</i>	<i>Hypothetical Stack Address</i>	<i>Value (Garbage Data)</i>	<i>Pointer Status</i>
<i>update[0]</i>	<i>0x7fff0000 to 0x7fff0007</i>	<i>0x4a3f8921bce45099</i>	<i>Wild Pointer (Points to a random, invalid address)</i>
<i>update[1]</i>	<i>0x7fff0008 to 0x7fff000f</i>	<i>0x0000000000000002</i>	<i>Wild Pointer</i>
<i>update[2]</i>	<i>0x7fff0010 to 0x7fff0017</i>	<i>0x7ffee1245a00881c</i>	<i>Wild Pointer</i>
<i>update[3]</i>	<i>0x7fff0018 to 0x7fff001f</i>	<i>0x0000000000000000</i>	<i>Accidentally NULL (Just by chance)</i>

More simpler way:

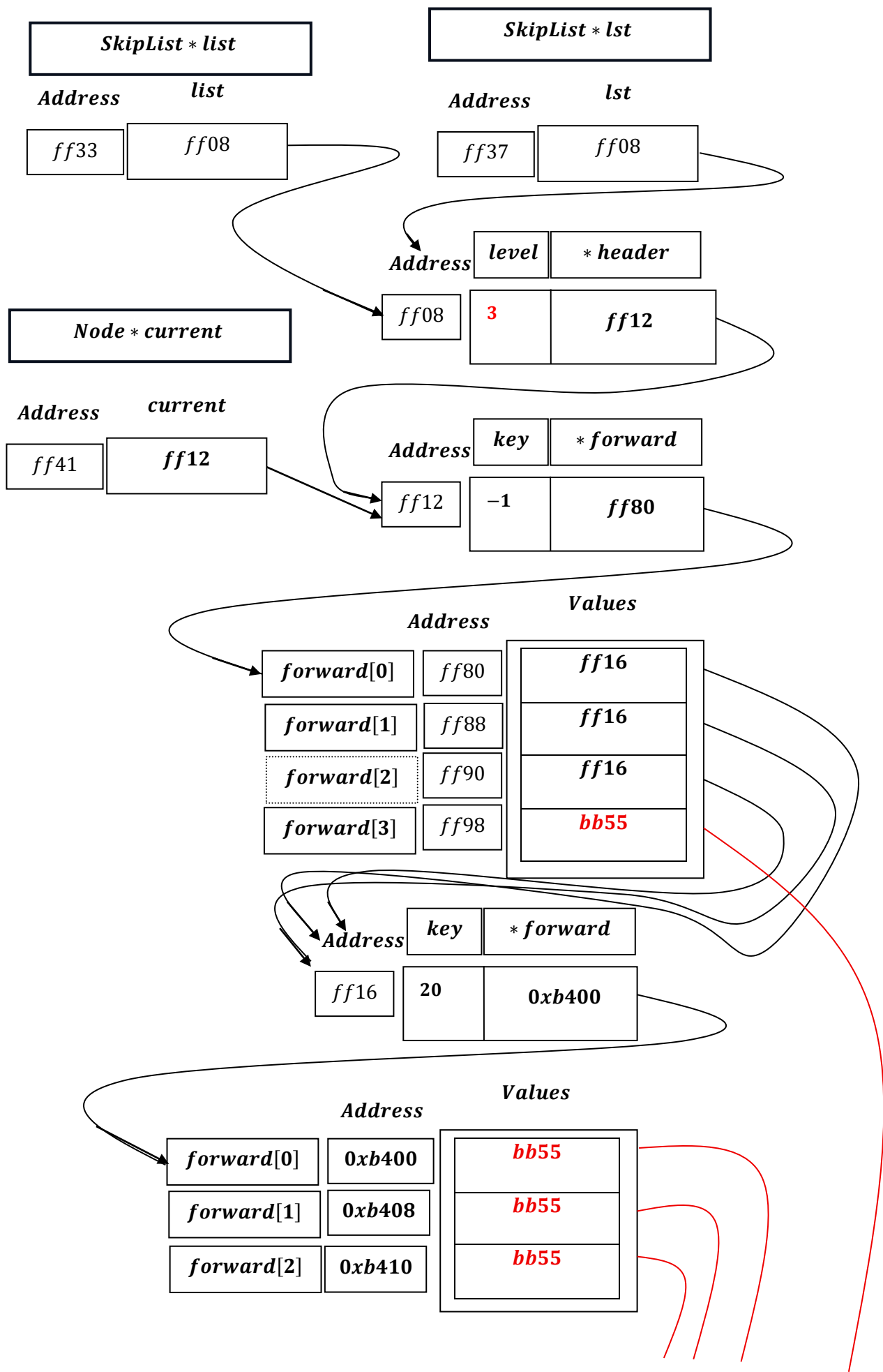
	<i>Address</i>	<i>Values</i>	
<i>update[0]</i>	<i>0x7fff0000</i>	<i>0x4a3f8921bce45099</i>	<i>[8 bytes]</i>
<i>update[1]</i>	<i>0x7fff0008</i>	<i>0x0000000000000002</i>	<i>[8 bytes]</i>
<i>update[2]</i>	<i>0x7fff0010</i>	<i>0x7ffee1245a00881c</i>	<i>[8 bytes]</i>
<i>update[3]</i>	<i>0x7fff0018</i>	<i>0x0000000000000000</i>	<i>[8 bytes]</i>

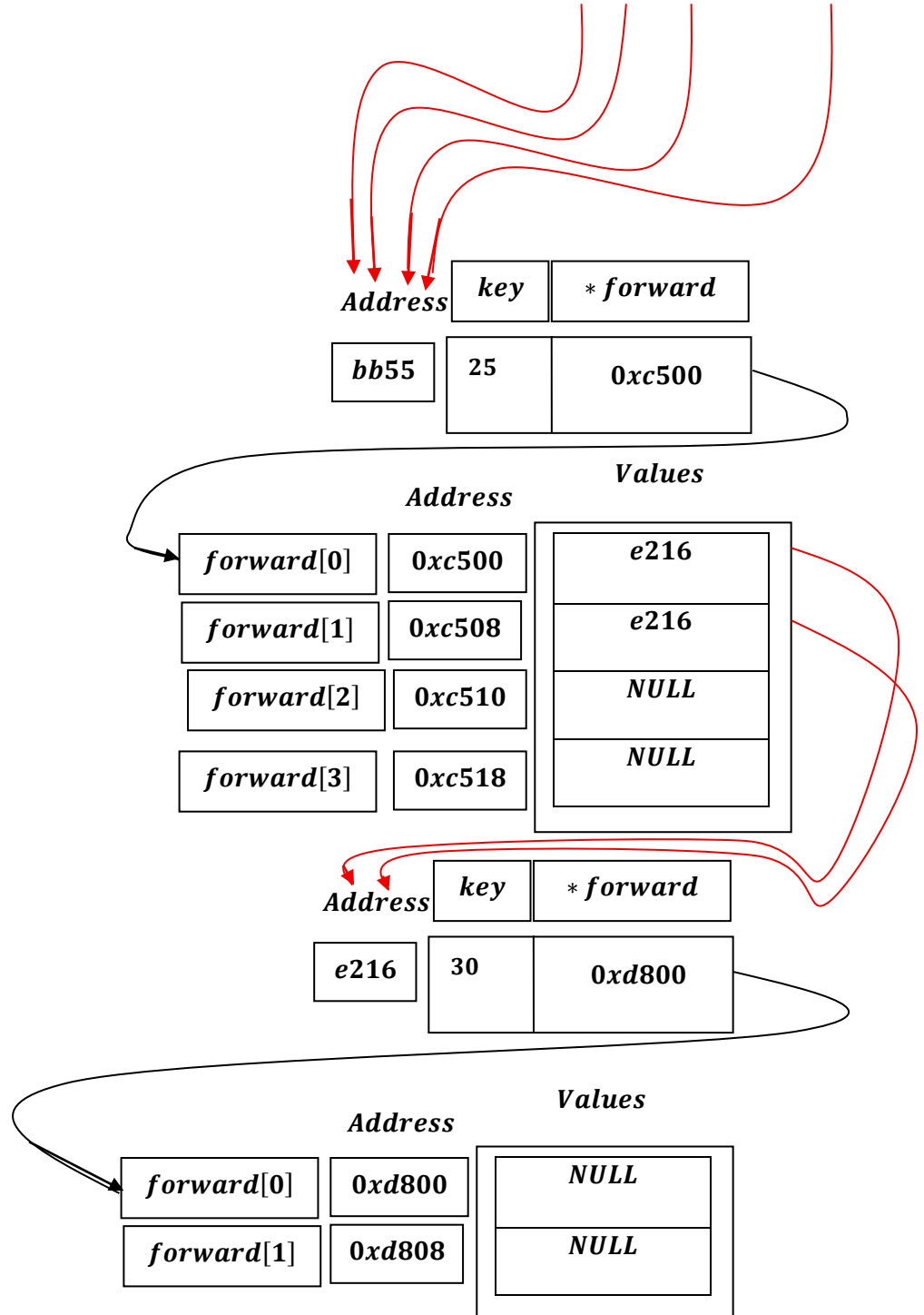
```
memset(update, 0, sizeof(struct Node *) * (MAX_LEVEL + 1));
```

“memset” stands for Memory Set. It takes a block of memory and rapidly fills it with a specific value. Here `0` or NULL.

	Address	Values
update[0]	0x7fff0000	NULL
update[1]	0x7fff0008	NULL
update[2]	0x7fff0010	NULL
update[3]	0x7fff0018	NULL

Here we kept addresses same like: 0x7fff0000 , just for an example , but it may not be same , in reality.





	Address	Values
update[0]	0x7fff0000	NULL
update[1]	0x7fff0008	NULL
update[2]	0x7fff0010	NULL
update[3]	0x7fff0018	NULL

```

for (int i = lst->level; i >= 0; i--) {
    while (current->forward[i] != NULL && current->forward[i]->key <
key) {
        current = current->forward[i];
    }
    update[i] = current;
}

```

For : $i = lst \rightarrow level = 3 ; i \geq 0$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$

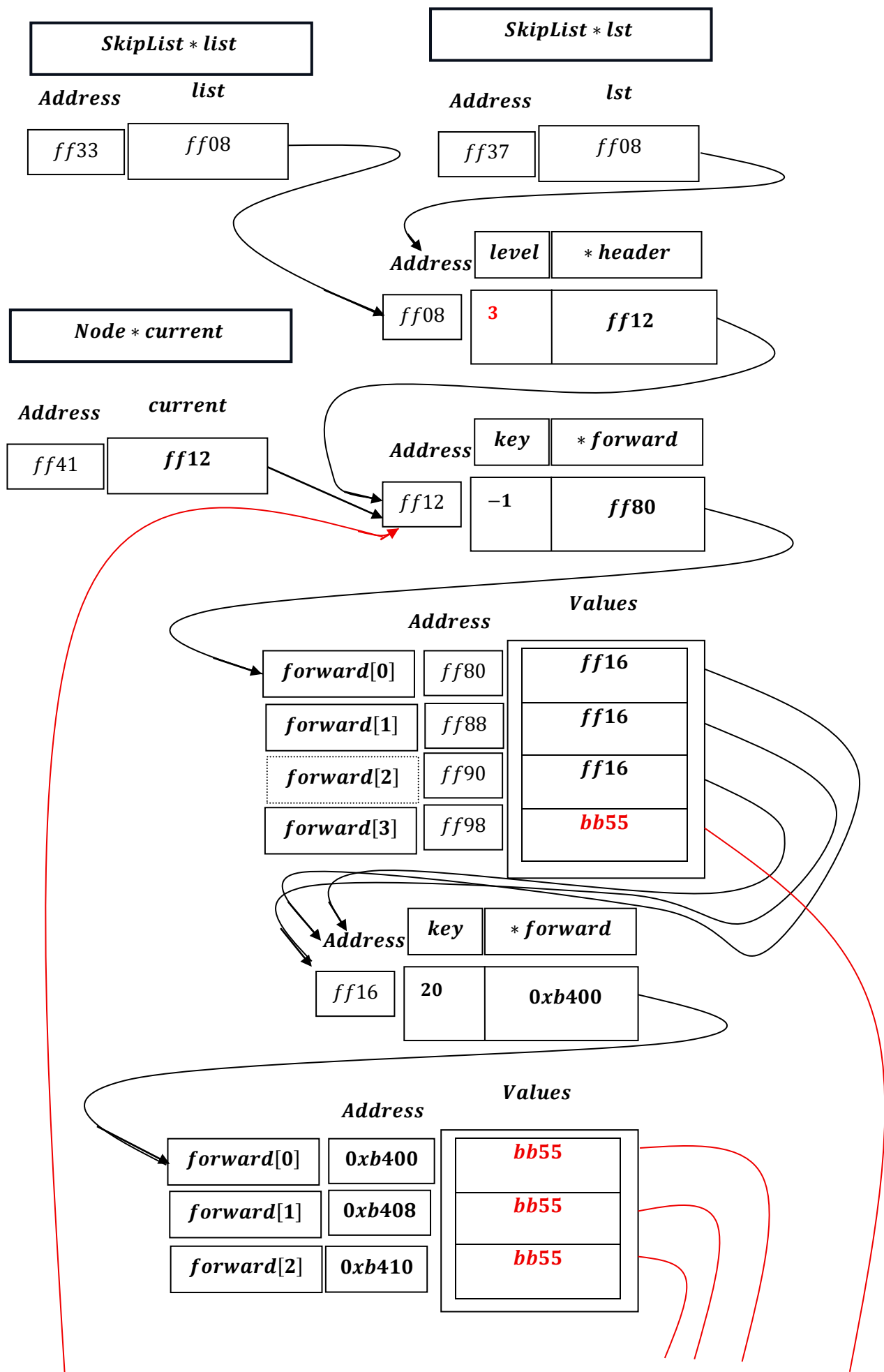
while: $current \rightarrow forward[3] = bb55 \neq NULL$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$ And

$current \rightarrow forward[3] \rightarrow key: 25 < key: 25$ $\left[\begin{array}{l} \text{Condition} \\ \text{is False} \end{array} \right]$

[while loop exits]

update[3] = current = ff12.

$i = i - 1 = 3 - 1 = 2.$



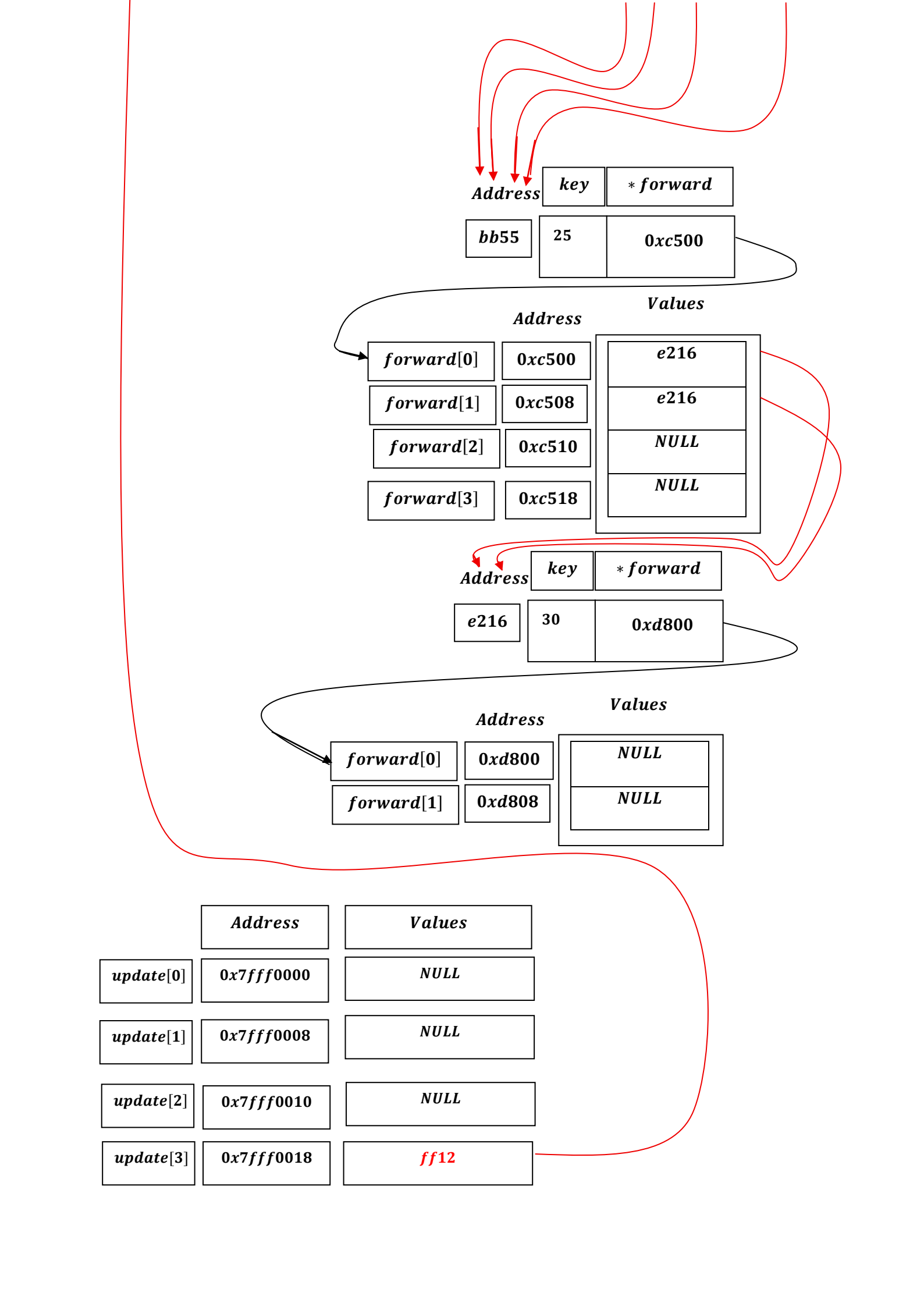
<i>Address</i>	<i>key</i>	<i>* forward</i>
<i>bb55</i>	25	0xc500

<i>Address</i>		<i>Values</i>
<i>forward[0]</i>	0xc500	e216
<i>forward[1]</i>	0xc508	e216
<i>forward[2]</i>	0xc510	NULL
<i>forward[3]</i>	0xc518	NULL

<i>Address</i>	<i>key</i>	<i>* forward</i>
<i>e216</i>	30	0xd800

<i>Address</i>		<i>Values</i>
<i>forward[0]</i>	0xd800	NULL
<i>forward[1]</i>	0xd808	NULL

	<i>Address</i>	<i>Values</i>
<i>update[0]</i>	0x7fff0000	NULL
<i>update[1]</i>	0x7fff0008	NULL
<i>update[2]</i>	0x7fff0010	NULL
<i>update[3]</i>	0x7fff0018	ff12



For : $i = 2 ; i \geq 0$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$

while: $current[ff12] \rightarrow forward[2] = ff16 \neq NULL$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$ And

$current[ff12] \rightarrow forward[2] \rightarrow key: 20 < key: 25$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$

$current = current \rightarrow forward[2] = ff16$

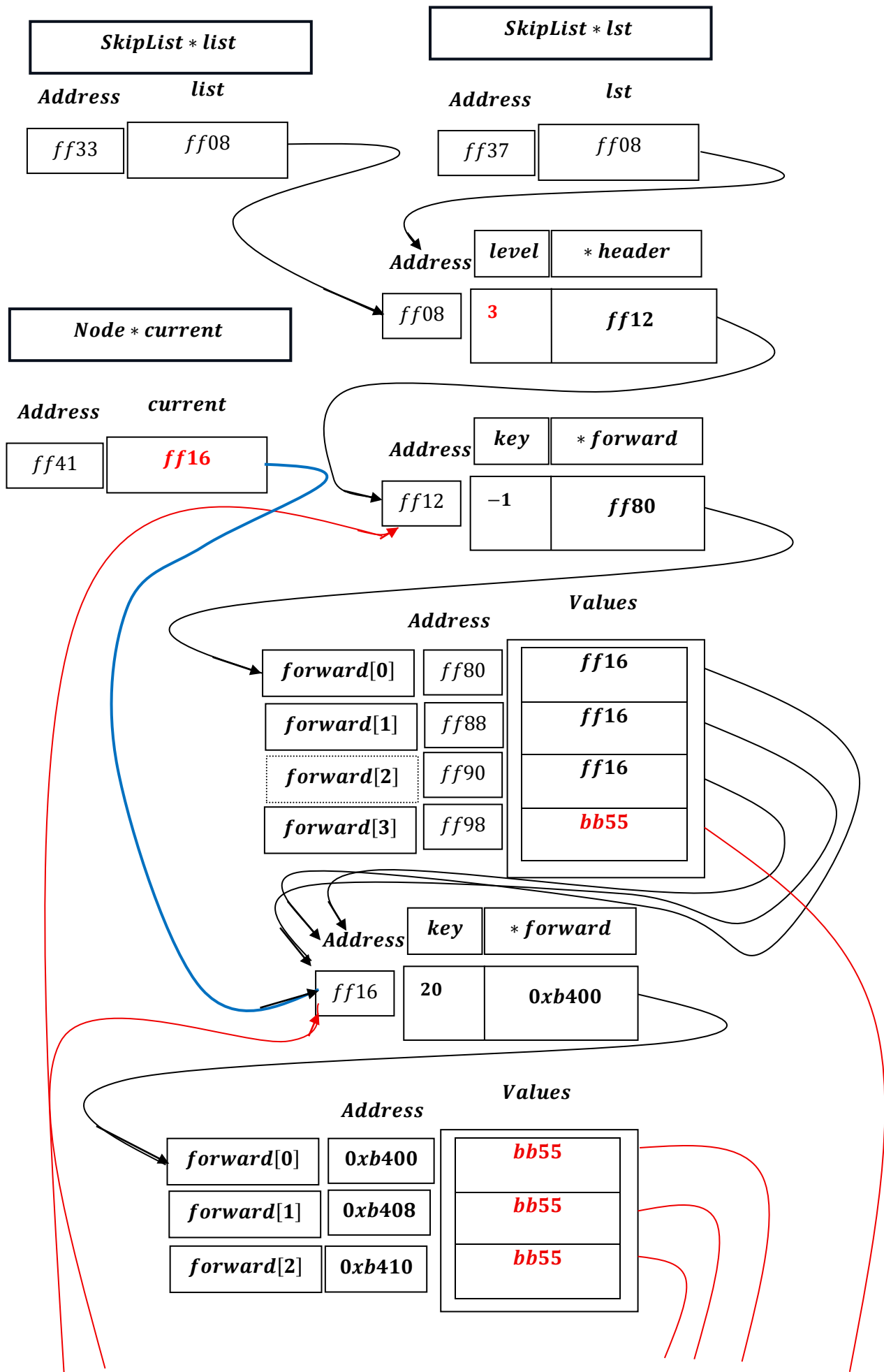
while: $current[ff16] \rightarrow forward[2] = bb55 \neq NULL$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$ And

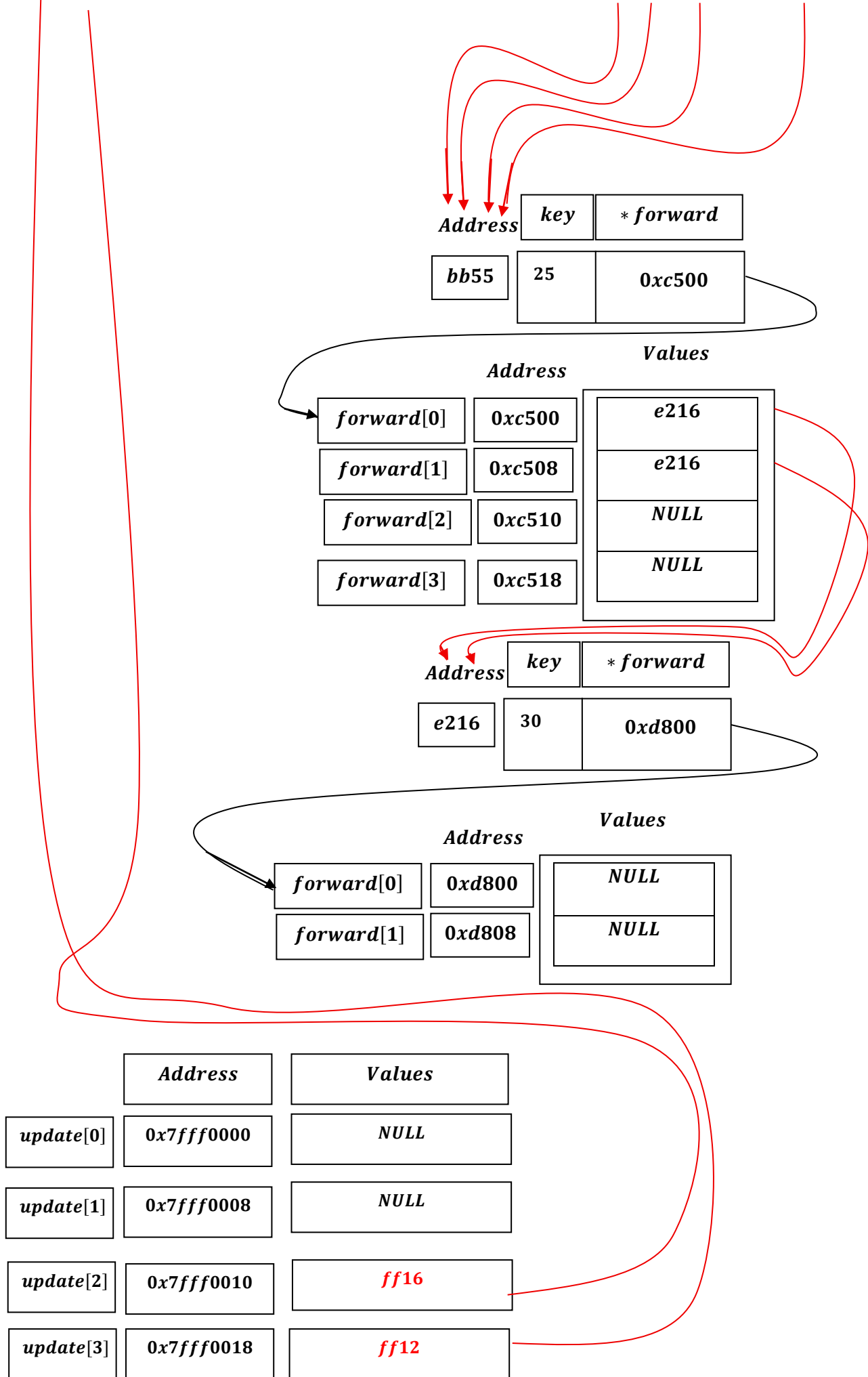
$current[ff16] \rightarrow forward[2] \rightarrow key: 25 < key: 25$ $\left[\begin{array}{l} \text{Condition} \\ \text{is False} \end{array} \right]$

[while loop exits]

$update[2] = current = ff16.$

$i = i - 1 = 2 - 1 = 1.$





For : $i = 1 ; i \geq 0$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$

while: $\text{current}[\text{ff16}] \rightarrow \text{forward}[1] = \text{bb55} \neq \text{NULL}$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$ And

$\text{current}[\text{ff16}] \rightarrow \text{forward}[1] \rightarrow \text{key: 25} < \text{key: 25}$ $\left[\begin{array}{l} \text{Condition} \\ \text{is False} \end{array} \right]$

[while loop exits]

$\text{update}[1] = \text{current} = \text{ff16}$.

$i = i - 1 = 1 - 1 = 0$.

For : $i = 0 ; i \geq 0$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$

while: $\text{current}[\text{ff16}] \rightarrow \text{forward}[0] = \text{bb55} \neq \text{NULL}$ $\left[\begin{array}{l} \text{Condition} \\ \text{is True} \end{array} \right]$ And

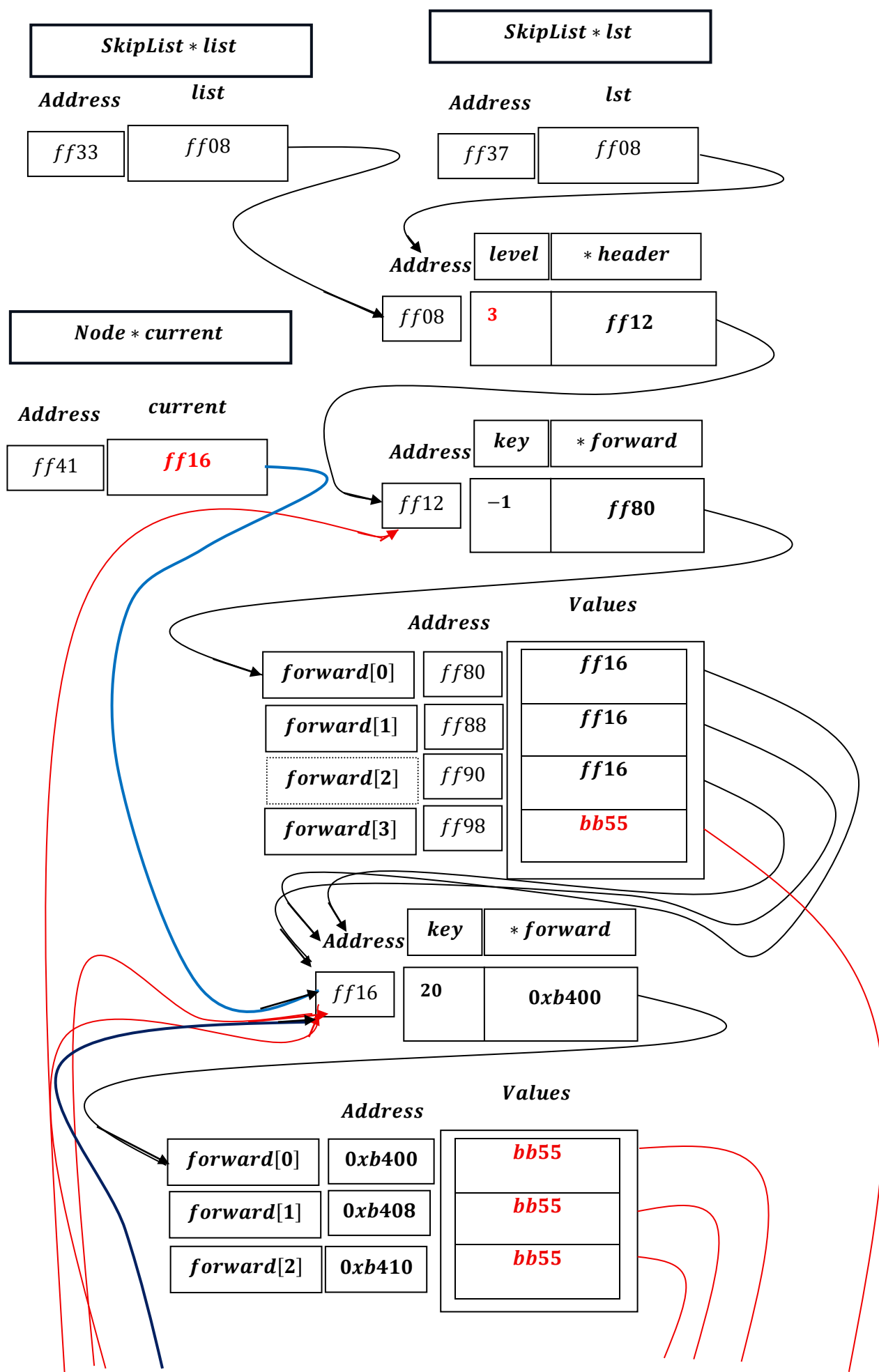
$\text{current}[\text{ff16}] \rightarrow \text{forward}[0] \rightarrow \text{key: 25} < \text{key: 25}$ $\left[\begin{array}{l} \text{Condition} \\ \text{is False} \end{array} \right]$

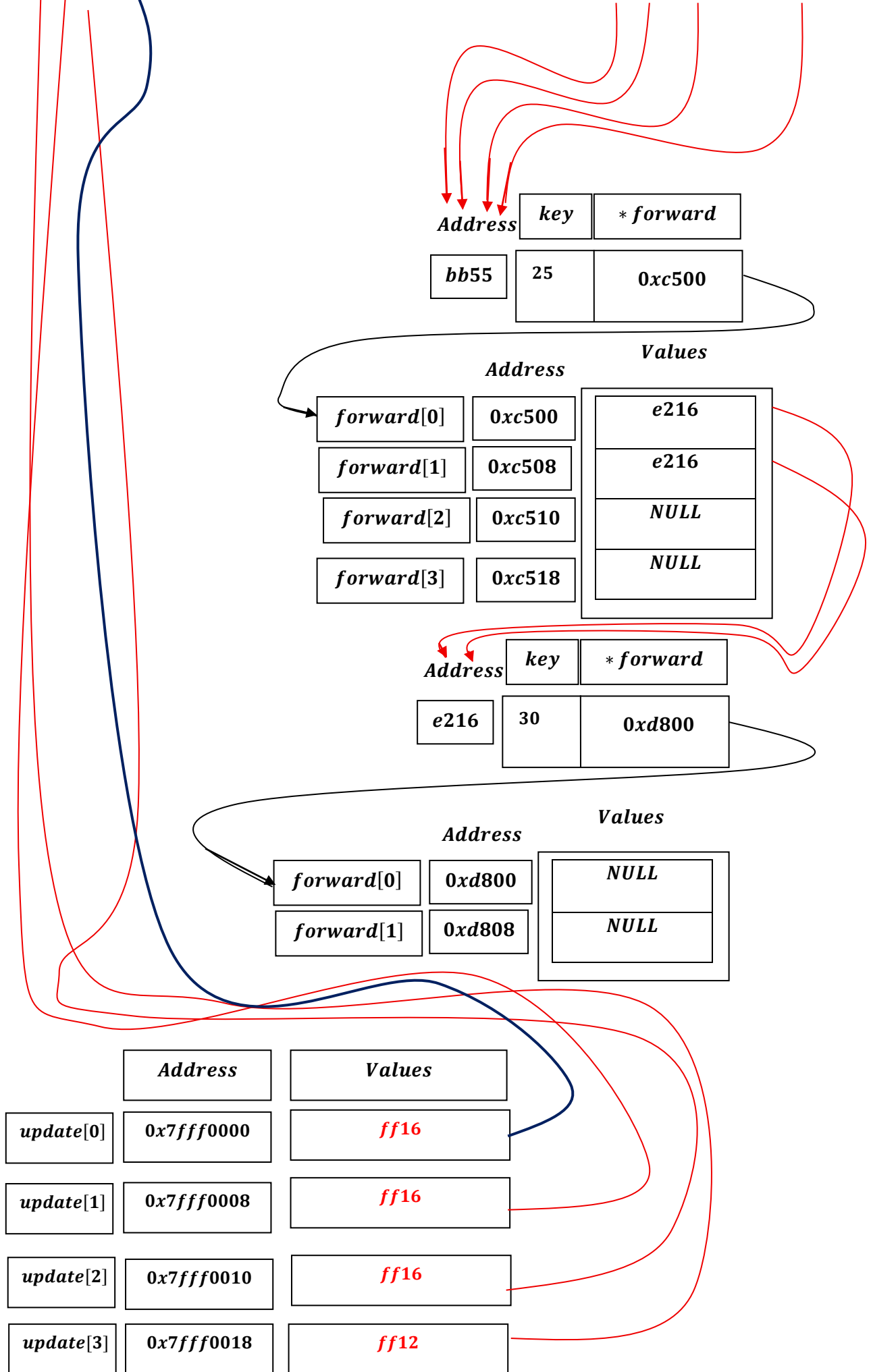
[while loop exits]

$\text{update}[0] = \text{current} = \text{ff16}$.

$i = i - 1 = 0 - 1 = -1$.

For : $i = -1 ; i \geq 0$ $\left[\begin{array}{l} \text{Condition} \\ \text{is False} \end{array} \right]$ $\rightarrow$ For loop exits.



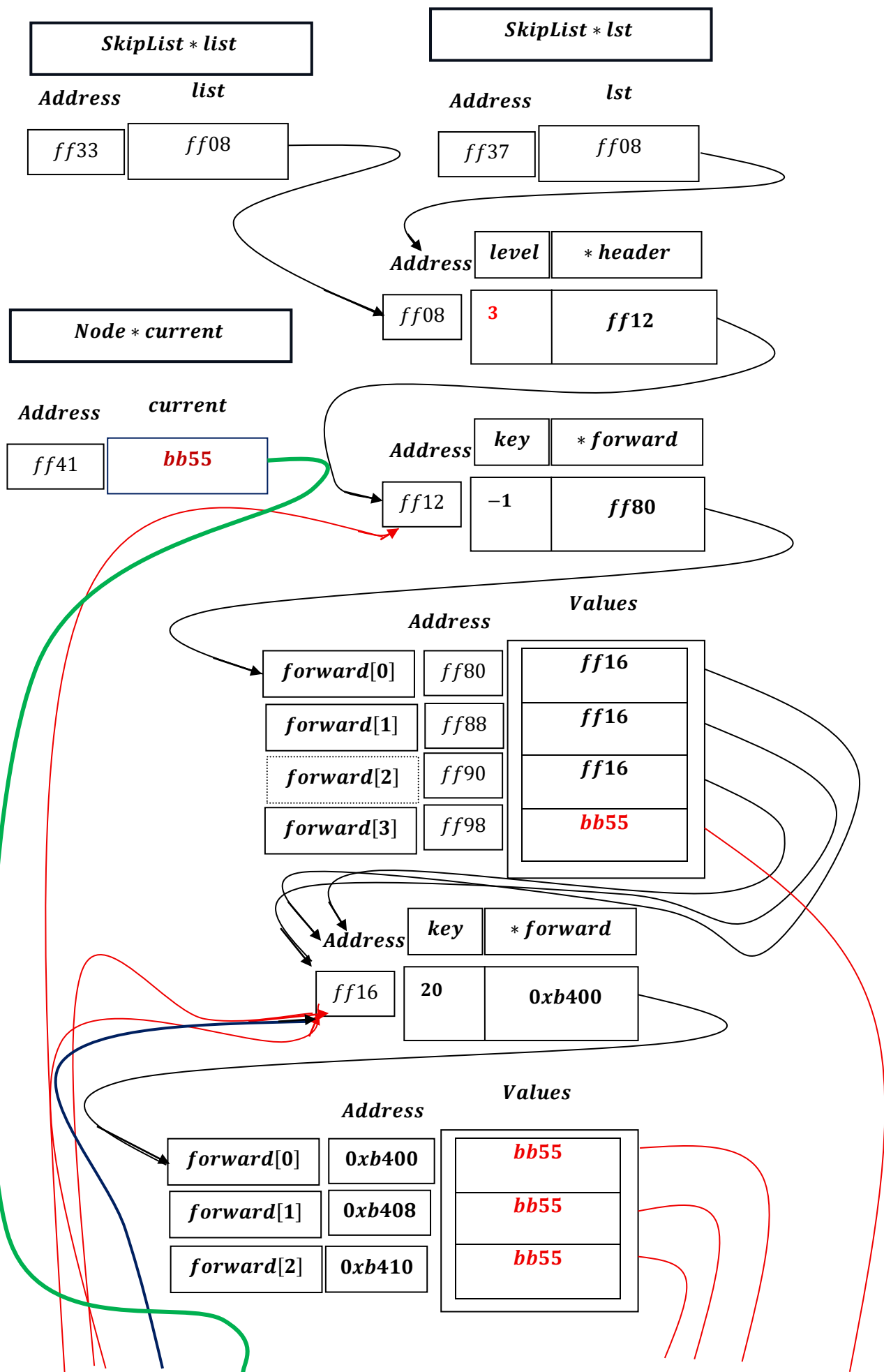


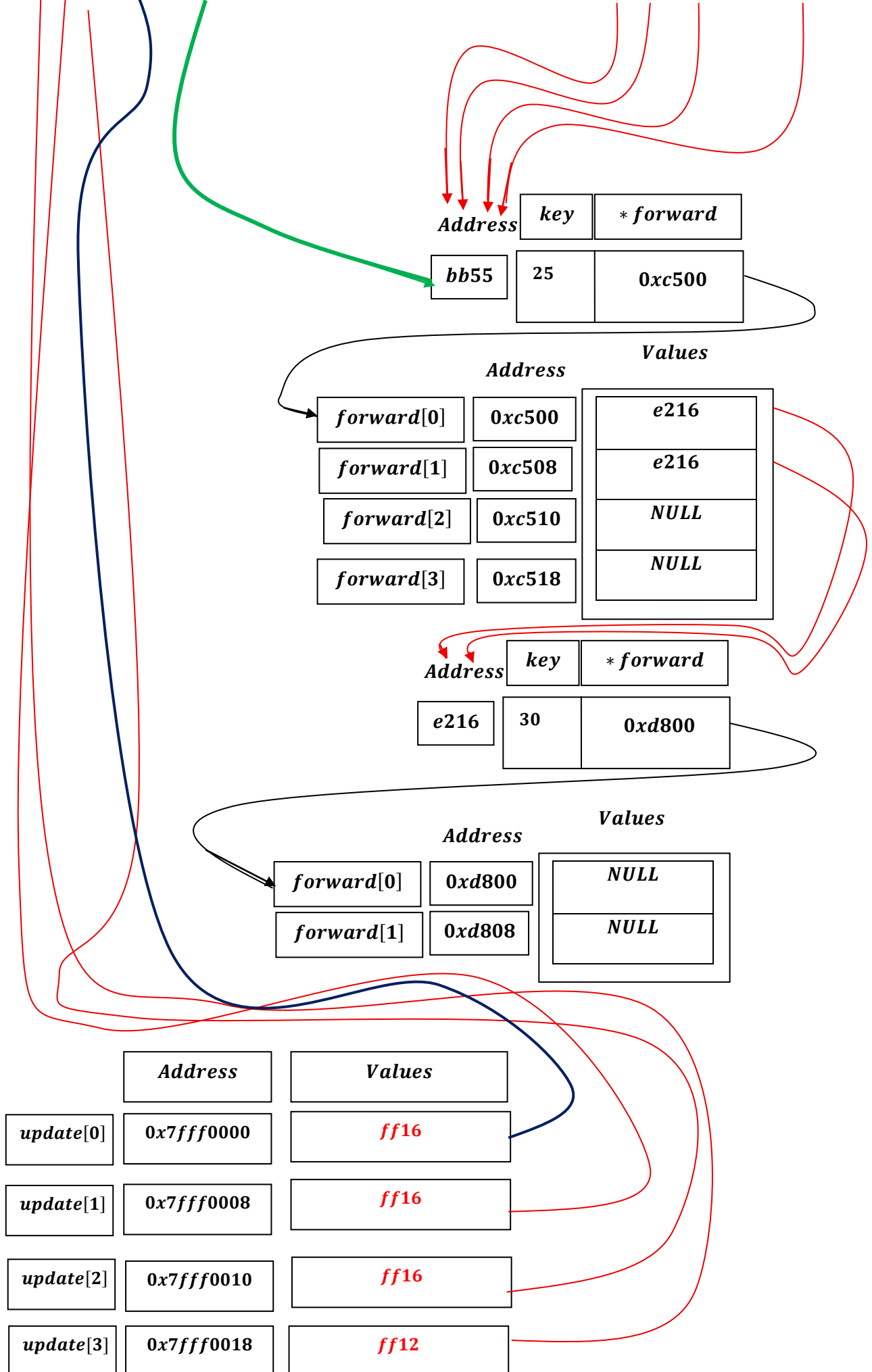
```
current = current->forward[0];
```

current = current → forward[0] = bb55

```
if (current == NULL || current->key != key) {...}
```

*current = ff16 ≠ NULL (false) OR current → key = 25 ≠ key = 25 (False),
hence if block terminates.*





*As we attempt to insert duplicate key it fails, hence ,
here in this program ,no duplicate keys are allowed.*

Can Skip Lists ever have duplicates?

As a general computer science data structure, a Skip List can be programmed to allow duplicates if the developer wants them to. To do that, we would just remove that:

`!= key`

check and allow the new node to be inserted right next to the existing one.

*However, standard sets and dictionaries (which skip lists are often used to build)
usually require unique keys, which is why the provided program is an implementation that
strictly blocks duplicates!*
